Instituto Profesional Virginio Gómez
Universidad de Concepción

Construyendo el Sistema de Personal IPVG

Semana 8: Encapsulamiento Avanzado y Herencia en .NET C# (Código 0568)

Ingeniería de Ejecución en Computación e Informática — Área de Informática

Gemini Notebook



El Desafío Arquitectónico: Sistema de Personal Sede IPVG

Objetivo: Construir un modelo robusto para gestionar Docentes, Administrativos y Directivos.

1. 100% Seguro
(Protección de datos sensibles).

2. Escalable
(Plantilla dinámica que crece día a día).

3. Sin Redundancia
(Principio DRY - Don't Repeat Yourself).

Gemini Notebook

Organización Moderna: File-Scoped Namespaces

Menos ruido visual, mayor claridad estructural.

Antes - C# Clásico

```
namespace SistemaPersonalIPVG
{
    public class Docente
    {
        // Código anidado
    }
}
```



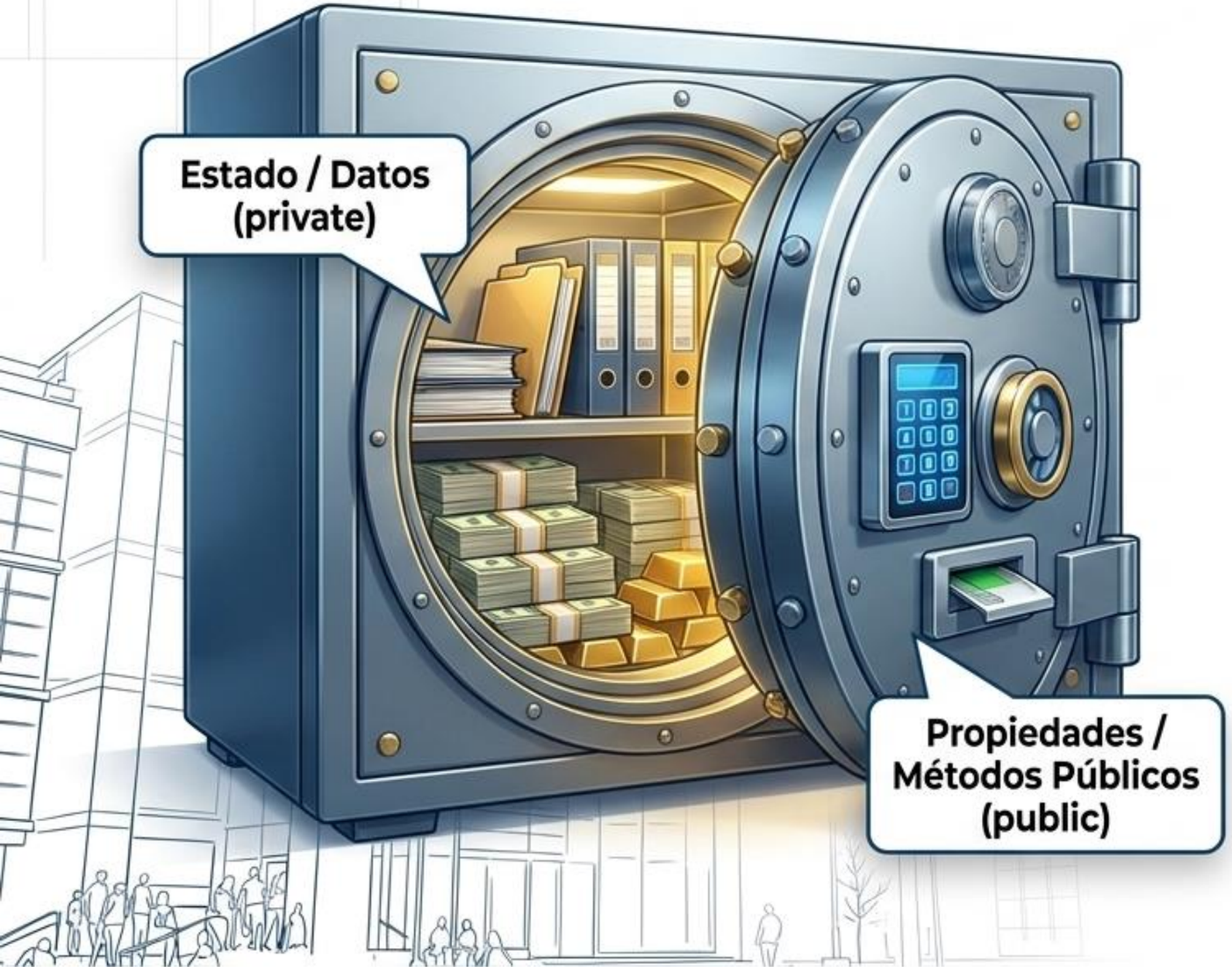
Ahora - C# 10+

```
namespace SistemaPersonalIPVG;

public class Docente
{
    // Código limpio
}
```

Gemini Notebook

El Principio de Encapsulamiento: Proteger la Integridad



¿Por qué ocultar los datos?

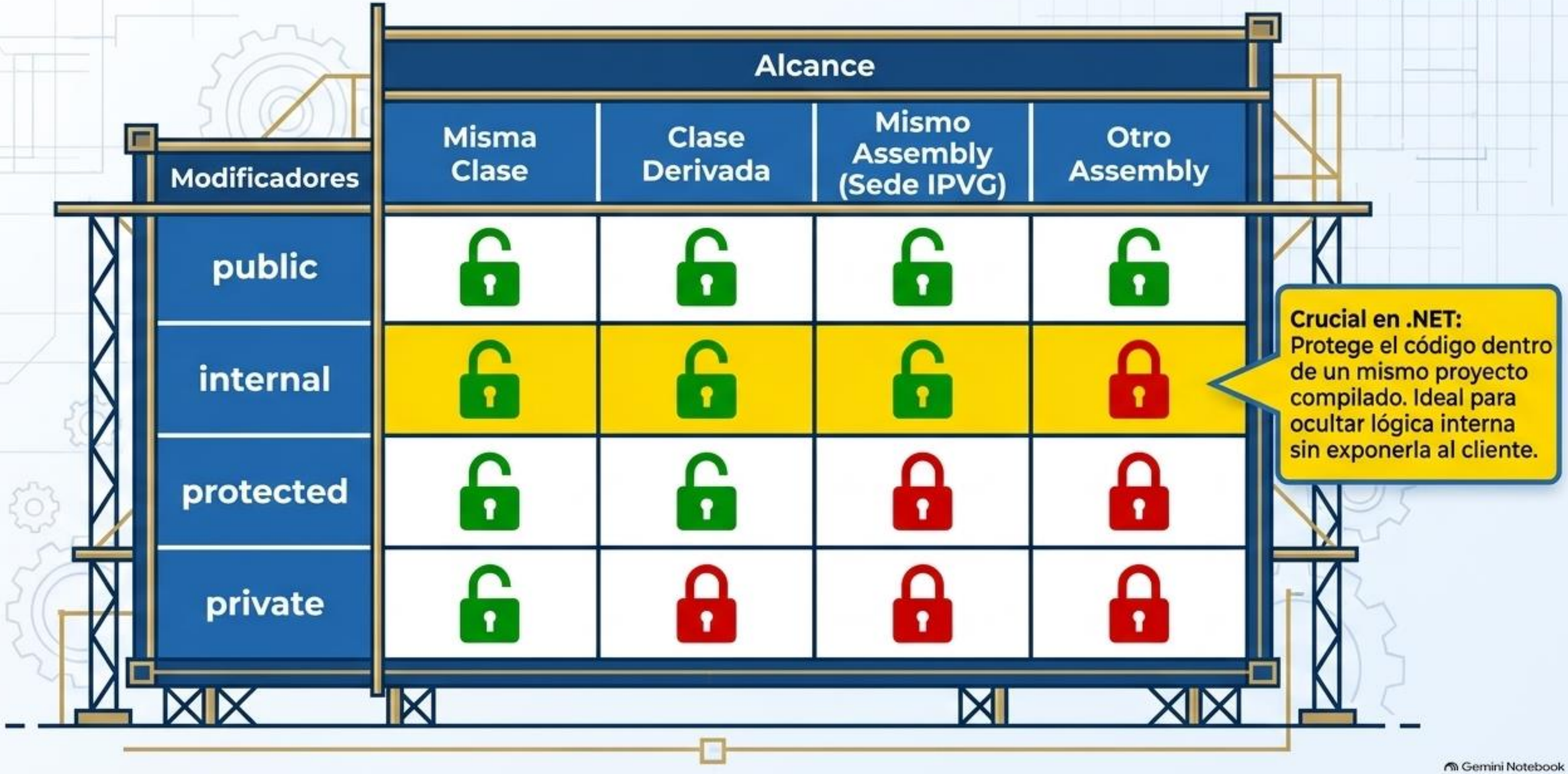
Para controlar el estado y validar reglas de negocio estandarizadas.

```
private decimal sueldoBase;

public decimal SueldoBase
{
    set
    {
        if (value < 0)
            throw new Exception("El sueldo no puede ser negativo");
        sueldoBase = value;
    }
}
```

Gemini Notebook

Matriz de Seguridad C#: Modificadores de Acceso



Gemini Notebook

El Problema de la Escalabilidad

La Sede IPVG acaba de contratar 5 nuevos docentes a mitad de semestre. Si usamos arrays tradicionales, nuestro sistema requiere recopiado manual o colapsa por IndexOutOfRangeException.



Arreglos Rígidos (Empleado[])

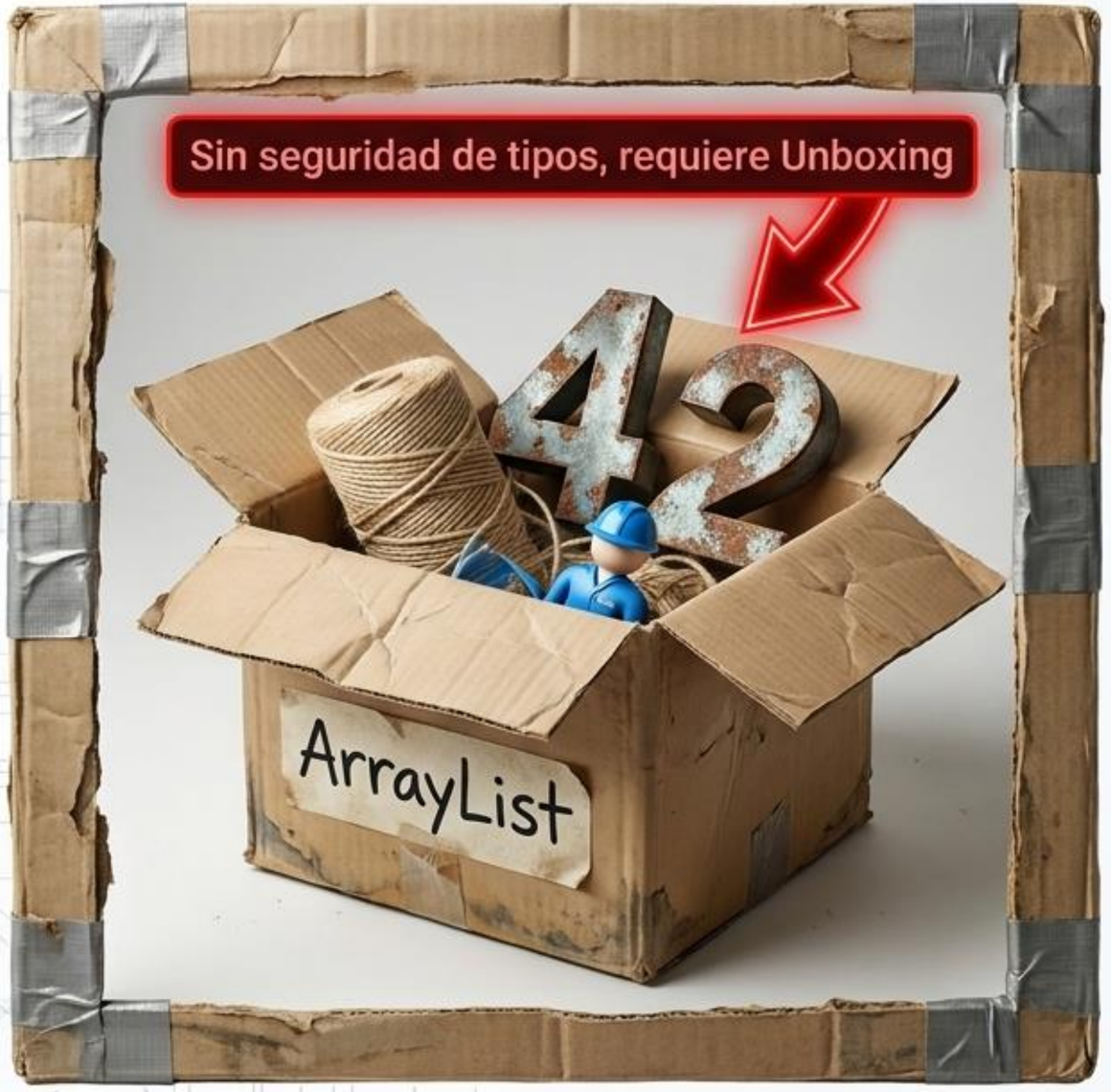


Colecciones Dinámicas (List<T>)

Gemini Notebook

La Evolución en.NET: ArrayList vs List<T>

El estándar moderno exige colecciones genéricas para evitar errores en tiempo de ejecución.





Gemini Notebook

Implementando List<T> en el Sistema IPVG

Gestión dinámica de la memoria sin sacrificar la **legibilidad**.

Dark Mode IDE

```
List<Empleado> nominas = new();

nominas.Add(nuevoDocente);

foreach (var empleado in nominas)
{
    Console.WriteLine(empleado.Nombre);
}
```

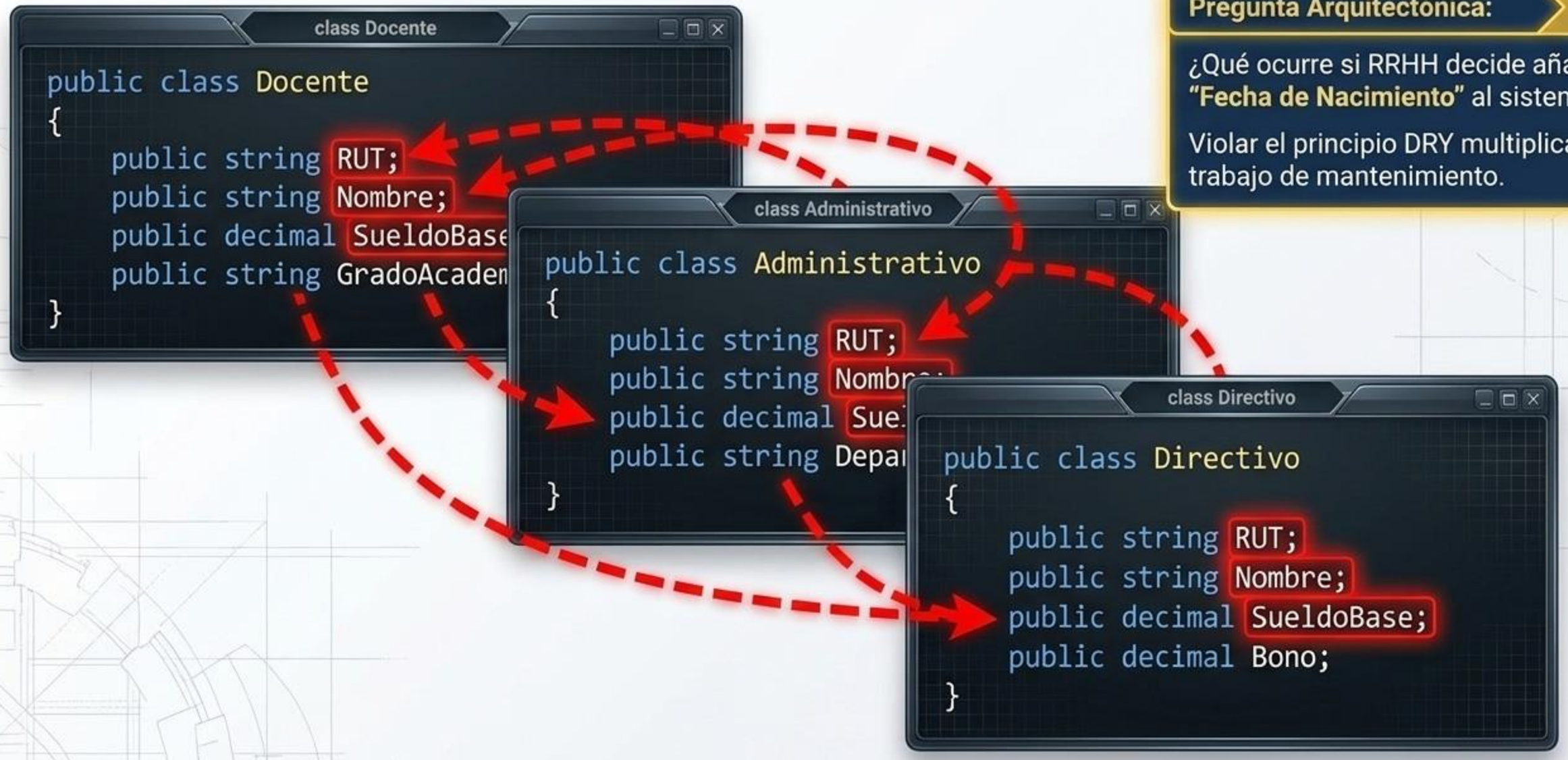
Target-typed new, código más limpio

Crece dinámicamente

Iteración segura de tipos

Gemini Notebook

El Anti-Patrón: La Trampa de la Redundancia



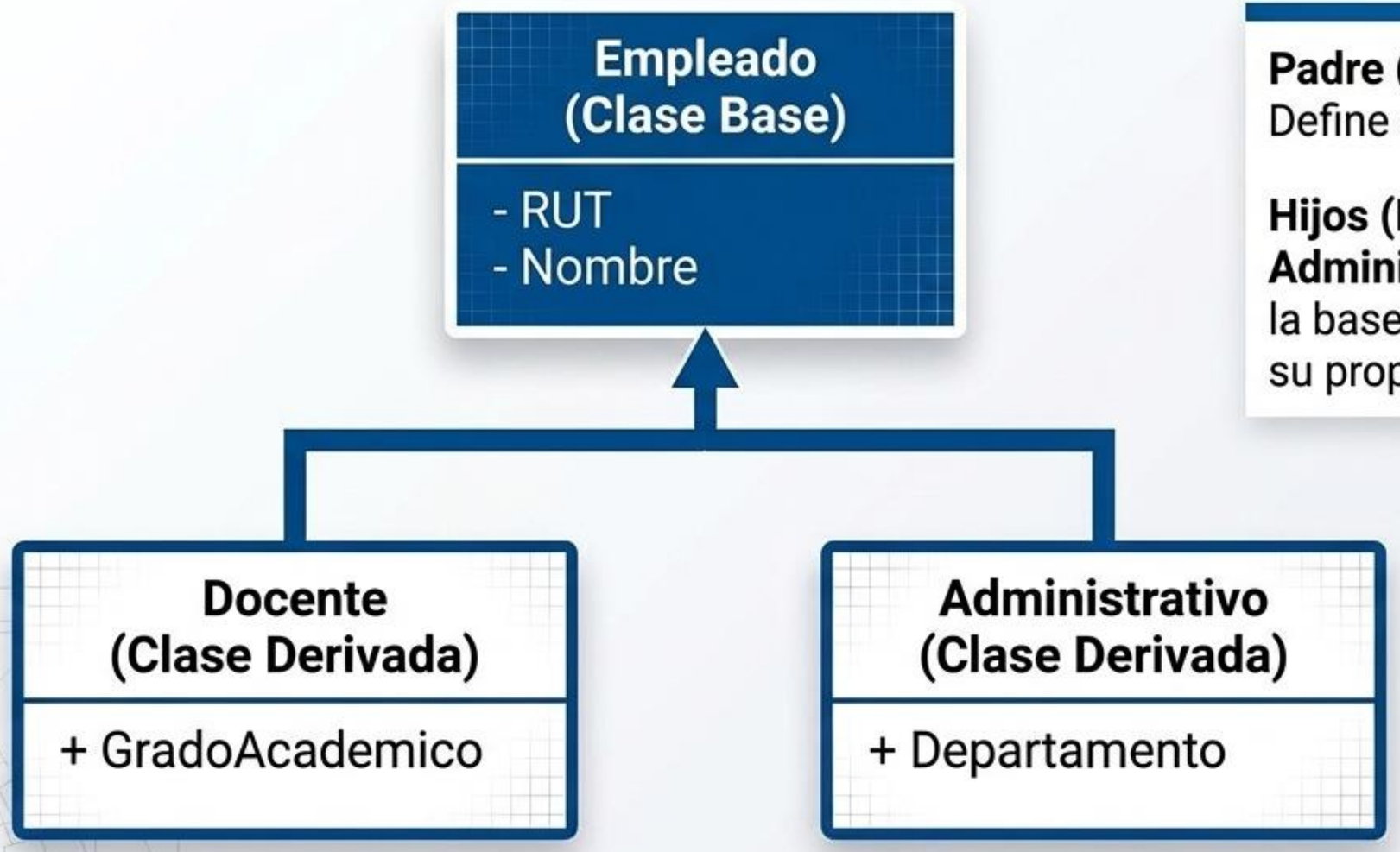
Pregunta Arquitectónica:

¿Qué ocurre si RRHH decide añadir **"Fecha de Nacimiento"** al sistema?

Violar el principio DRY multiplica el trabajo de mantenimiento.

Gemini Notebook

Introducción a la Herencia: La Relación 'Es Un' (Is-A)



Padre (Empleado): Define atributos comunes.

Hijos (Docentes, Administrativos): Heredan la base estructural y añaden su propia especialización.

Gemini Notebook

Sintaxis de Herencia en C# (.NET)

```
public class Docente : Empleado
{
    public string Grado { get; set; }

    public Docente(string rut, string nombre, string grado) : base(rut, nombre)
    {
        Grado = grado;
    }
}
```

1

1. El Contrato:

El operador El operador ':' establece la relación de herencia. Docente ahora posee todos los métodos y propiedades de Empleado.

2

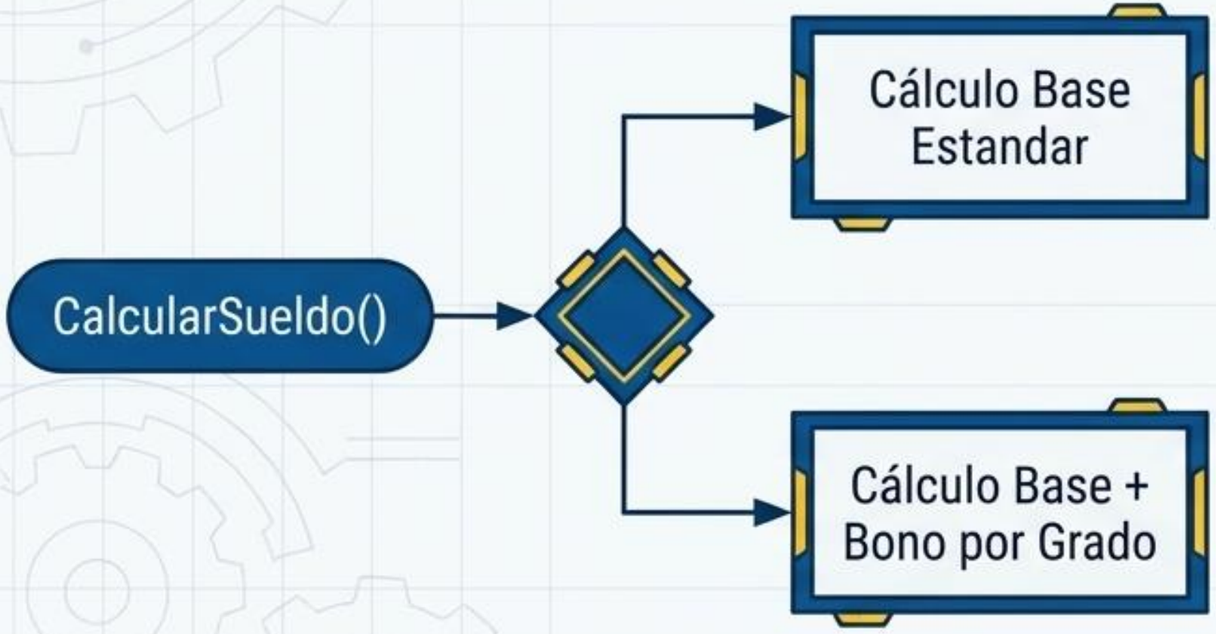
2. El Constructor Padre:

2. El Constructor Padre: La palabra clave **`base`** envía inmediatamente los parámetros hacia arriba, al constructor de la clase superior, antes de construir el objeto actual.

Gemini Notebook

Comportamiento Especializado: Virtual y Override

Un mismo método, diferentes respuestas según el tipo de empleado.



Clase Base: Empleado

```
public virtual decimal CalcularSueldo()
{
    return SueldoBase;
}
```

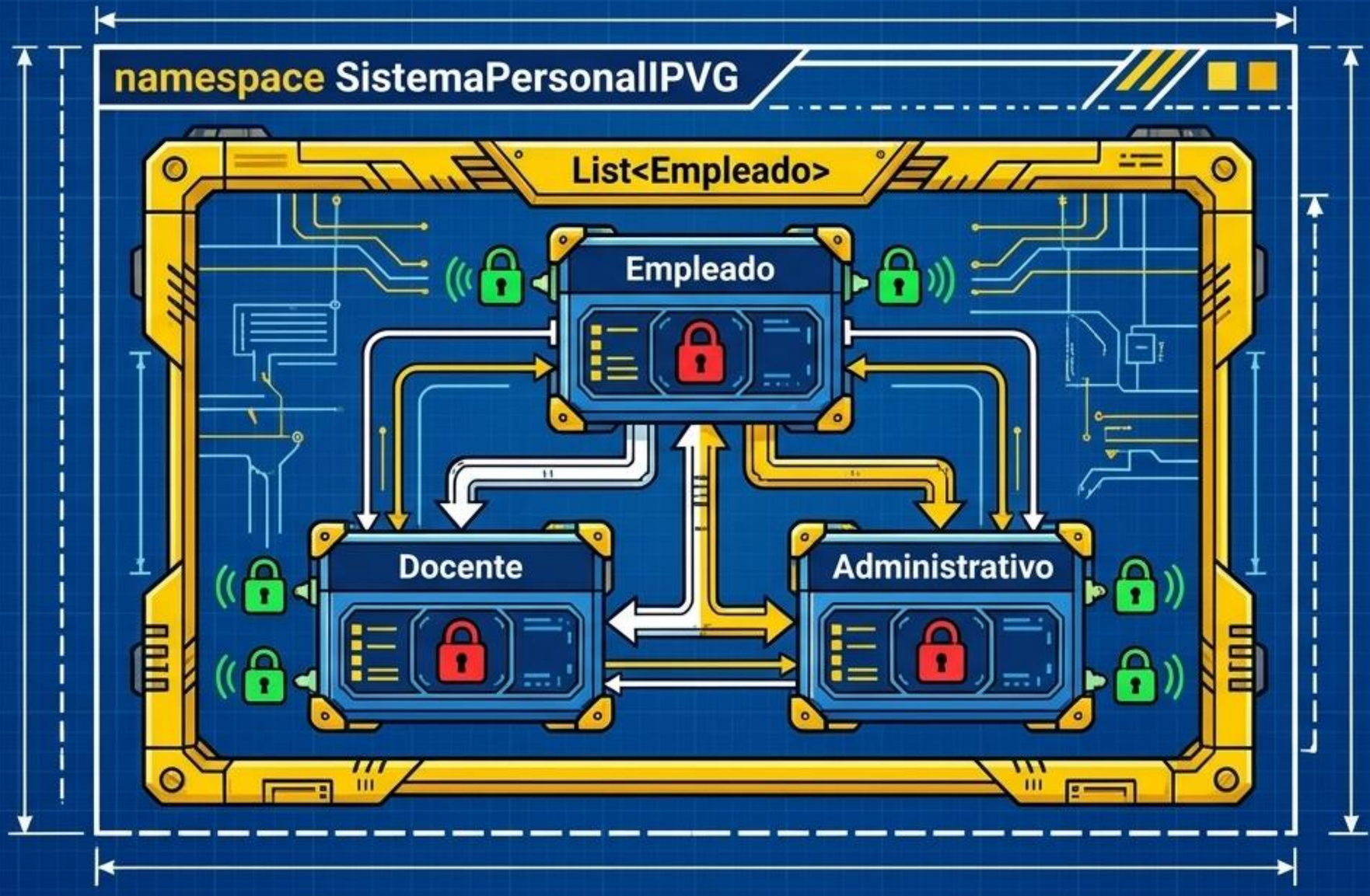
Clase Derivada: Docente

```
public override decimal CalcularSueldo()
{
    return base.CalcularSueldo() + BonoGrado;
}
```

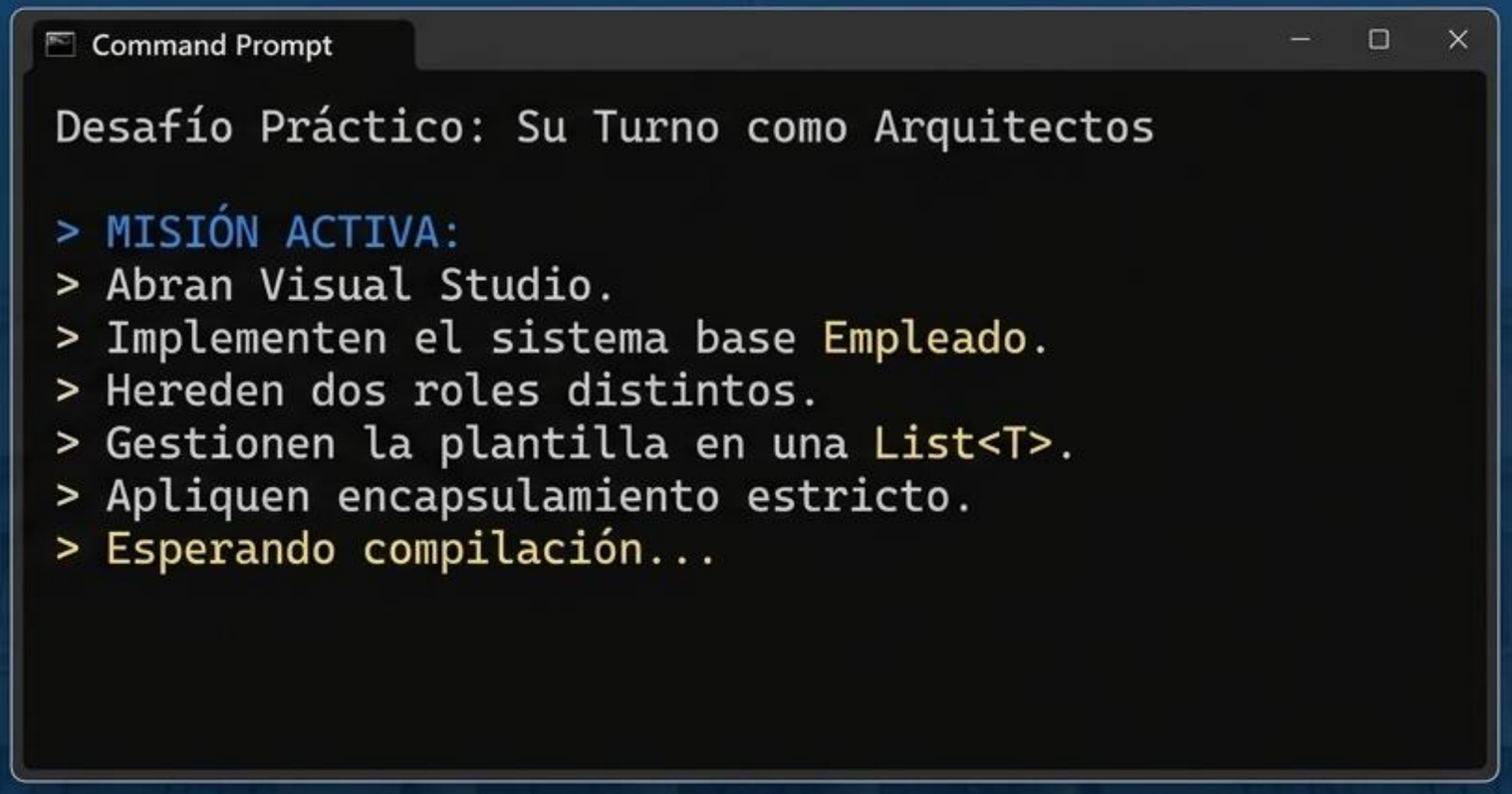
Gemini Notebook

Síntesis Arquitectónica: El Sistema IPVG Terminado

Integrando Namespaces, Colecciones, Herencia y Seguridad.



Gemini Notebook



El buen código resuelve problemas.
La buena arquitectura previene desastres.

ACREDITADO
ACREDITADO
Área
Informática
IPVG
Gemini Notebook